

História da Linguagem SQL

A linguagem SQL (Structured Query Language) surgiu na década de 1970.

Ela foi criada por pesquisadores da empresa IBM, com base no modelo relacional proposto por Edgar F. Codd em 1970.

Inicialmente, a linguagem se chamava SEQUEL (Structured English Query Language). Depois, o nome foi alterado para SQL, por questões de marca registrada.

Linha do Tempo

- 1970 – Edgar F. Codd publica o artigo sobre o modelo relacional.
- 1974 – A IBM desenvolve a linguagem SEQUEL.
- 1986 – A SQL se torna padrão oficial pela ANSI.
- 1987 – A ISO também adota SQL como padrão internacional.

Desde então, a SQL se tornou a principal linguagem utilizada para manipular bancos de dados relacionais no mundo.

Hoje, praticamente todos os SGBDs relacionais utilizam SQL, como:

- MySQL
- Oracle Database
- Microsoft SQL Server
- PostgreSQL

PostgreSQL

◇ O que é?

O PostgreSQL é um Sistema de Gerenciamento de Banco de Dados Relacional (SGBD), de código aberto (open source), extremamente robusto e avançado.

Ele é considerado um SGBD objeto-relacional, pois além de trabalhar com tabelas relacionais, também suporta recursos mais avançados como:

- Tipos personalizados
- Funções
- Procedimentos
- Herança de tabelas
- JSON
- Controle transacional completo (ACID)

História do PostgreSQL

O PostgreSQL tem origem no projeto POSTGRES, desenvolvido na Universidade da Califórnia em Berkeley, liderado pelo professor Michael Stonebraker.

- 1986 – Início do projeto POSTGRES.
- 1994 – Adição do suporte à linguagem SQL.
- 1996 – O nome passa a ser oficialmente PostgreSQL.

Desde então, o PostgreSQL evoluiu muito e hoje é um dos bancos de dados mais utilizados no mundo, principalmente em sistemas que exigem:

- Alta confiabilidade
- Integridade de dados
- Grande volume de informações

Por que usar PostgreSQL?

- ✓ Gratuito e open source
- ✓ Alta performance
- ✓ Seguro
- ✓ Muito estável
- ✓ Amplamente utilizado no mercado

APOSTILA COMPLETA – SQL + PostgreSQL (Didática e Profissional)

NÍVEL 1 — FUNDAMENTOS

1. SELECT

- ◇ O que faz?

Seleciona (busca) dados dentro de uma tabela.

É o comando mais usado em SQL.

-
- ◇ Exemplo 1

SELECT nome FROM funcionarios;

Busca apenas a coluna nome.

◇ Exemplo 2

```
SELECT * FROM funcionarios;
```

Busca todas as colunas.

 Atividade:

Crie uma tabela clientes e faça um SELECT mostrando apenas nome e cidade.

2. WHERE

◇ O que faz?

Filtra os dados retornados pelo SELECT.

Sem WHERE → vem tudo

Com WHERE → vem apenas o que atende a condição

◇ Exemplo 1

```
SELECT * FROM funcionarios  
WHERE salario > 3000;
```

◇ Exemplo 2

```
SELECT * FROM funcionarios  
WHERE departamento_id = 2 AND salario > 2000;
```

 Atividade:

Liste funcionários com salário entre 2000 e 5000.

3. ORDER BY

◇ O que faz?

Ordena os resultados.

ASC → crescente

DESC → decrescente

◇ Exemplo 1

```
SELECT nome, salario  
FROM funcionarios  
ORDER BY salario DESC;
```

◇ Exemplo 2

```
SELECT nome  
FROM funcionarios  
ORDER BY nome ASC;
```

 Atividade:

Liste funcionários do menor para o maior salário.

 4. LIMIT

◇ O que faz?

Limita a quantidade de registros retornados.

◇ Exemplo

```
SELECT * FROM funcionarios  
LIMIT 5;
```

 Atividade:

Mostre apenas os 3 salários mais altos.

 5. DISTINCT

◇ O que faz?

Remove valores repetidos.

◇ Exemplo

```
SELECT DISTINCT departamento_id  
FROM funcionarios;
```

 Atividade:

Liste cidades sem repetir nomes.

NÍVEL 2 — DML (Manipulação de Dados)

6. INSERT

- ◇ O que faz?

Insere novos registros na tabela.

- ◇ Exemplo 1

```
INSERT INTO funcionarios (nome, salario)
VALUES ('Carlos', 3000);
```

- ◇ Exemplo 2

```
INSERT INTO funcionarios (nome, salario)
VALUES
('Ana', 4000),
('João', 3500);
```

Atividade:

Insira 5 funcionários diferentes.

7. UPDATE

- ◇ O que faz?

Atualiza dados existentes.

 Sem WHERE altera tudo.

- ◇ Exemplo

```
UPDATE funcionarios
SET salario = salario * 1.10
WHERE id = 1;
```

Atividade:

Aumente 5% do salário de todos os funcionários do departamento 2.

8. DELETE

◇ O que faz?

Remove registros.

◇ Exemplo

```
DELETE FROM funcionarios  
WHERE id = 5;
```

 Atividade:

Delete funcionários com salário menor que 1500.

NÍVEL 3 — DDL (Estrutura)

9. CREATE DATABASE

◇ O que faz?

Cria um novo banco de dados.

```
CREATE DATABASE empresa_db;
```

 Atividade:

Crie um banco chamado loja_db.

10. CREATE TABLE

◇ O que faz?

Cria uma tabela.

```
CREATE TABLE funcionarios (  
  id SERIAL PRIMARY KEY,  
  nome VARCHAR(100),  
  salario NUMERIC(10,2)  
);
```

 Atividade:

Crie tabela produtos.

11. ALTER TABLE

◇ O que faz?

Altera estrutura da tabela.

Adicionar coluna:

```
ALTER TABLE funcionarios  
ADD COLUMN email VARCHAR(100);
```

Alterar tipo:

```
ALTER TABLE funcionarios  
ALTER COLUMN salario TYPE NUMERIC(12,2);
```

 Atividade:

Adicione coluna data_admissao.

12. DROP

◇ O que faz?

Exclui tabela ou banco.

```
DROP TABLE funcionarios;
```

 Atividade:

Crie uma tabela teste e depois apague.

NÍVEL 4 — RELACIONAMENTOS

13. PRIMARY KEY

◇ O que faz?

Identifica unicamente cada registro.

id SERIAL PRIMARY KEY

 Atividade:

Crie tabela com PK.

 14. FOREIGN KEY

◇ O que faz?

Cria relacionamento entre tabelas.

FOREIGN KEY (departamento_id)
REFERENCES departamentos(id)

 Atividade:

Relacione funcionários a departamentos.

 15. JOIN

◇ O que faz?

Une dados de duas ou mais tabelas.

◇ Exemplo 1 (INNER JOIN)

```
SELECT f.nome, d.nome_departamento
FROM funcionarios f
JOIN departamentos d
ON f.departamento_id = d.id;
```

◇ Exemplo 2 (LEFT JOIN)

```
SELECT f.nome, fp.salario
FROM funcionarios f
LEFT JOIN folha_pagamento fp
ON f.id = fp.funcionario_id;
```

 Atividade:

Liste funcionário + nome do departamento.

16. Funções Agregadas

- ◇ O que fazem?

Realizam cálculos sobre conjunto de linhas.

```
SELECT AVG(salario) FROM funcionarios;  
SELECT COUNT(*) FROM funcionarios;
```

Atividade:

Descubra o maior salário da empresa.

17. GROUP BY

- ◇ O que faz?

Agrupa registros para usar agregações.

```
SELECT departamento_id, AVG(salario)  
FROM funcionarios  
GROUP BY departamento_id;
```

Atividade:

Conte funcionários por departamento.

18. HAVING

- ◇ O que faz?

Filtra depois da agregação.

```
SELECT departamento_id, COUNT(*)  
FROM funcionarios  
GROUP BY departamento_id  
HAVING COUNT(*) > 2;
```

Atividade:

Mostre departamentos com média salarial maior que 3000.

NÍVEL 6 — INTERMEDIÁRIO

19. CASE WHEN

- ◇ O que faz?

Cria lógica condicional dentro da consulta.

```
SELECT nome,  
CASE  
  WHEN salario > 5000 THEN 'ALTO'  
  ELSE 'NORMAL'  
END  
FROM funcionarios;
```

Atividade:

Classifique funcionários como JUNIOR ou SENIOR.

20. COALESCE

- ◇ O que faz?

Substitui valores NULL.

```
SELECT COALESCE(comissao, 0)  
FROM funcionarios;
```

Atividade:

Mostre comissão como 0 quando for NULL.

21. SUBQUERY

- ◇ O que faz?

Consulta dentro de outra consulta.

```
SELECT nome  
FROM funcionarios  
WHERE salario > (  
  SELECT AVG(salario) FROM funcionarios  
);
```

 Atividade:

Liste quem ganha acima da média.

22. EXISTS

◇ O que faz?

Verifica se existe registro relacionado.

```
SELECT nome
FROM funcionarios f
WHERE EXISTS (
  SELECT 1
  FROM folha_pagamento fp
  WHERE fp.funcionario_id = f.id
);
```

 Atividade:

Liste funcionários que possuem registro na folha.

NÍVEL 7 — AVANÇADO

23. WINDOW FUNCTION

◇ O que faz?

Realiza cálculo sem perder as linhas originais.

```
SELECT nome,
salario,
SUM(salario) OVER (ORDER BY salario)
FROM funcionarios;
```

 Atividade:

Crie ranking salarial.

24. INDEX

◇ O que faz?

Melhora velocidade de busca.

```
CREATE INDEX idx_nome  
ON funcionarios(nome);
```

 Atividade:

Crie índice na coluna email.

25. VIEW

◇ O que faz?

Cria consulta salva como se fosse tabela.

```
CREATE VIEW vw_funcionarios AS  
SELECT nome, salario  
FROM funcionarios;
```

 Atividade:

Crie view juntando funcionário + departamento.

26. TRANSAÇÕES

◇ O que fazem?

Garantem segurança nas operações.

```
BEGIN;
```

```
UPDATE funcionarios  
SET salario = salario * 1.20;
```

```
ROLLBACK;
```

Ou:

```
COMMIT;
```

 Atividade:

Faça UPDATE e teste ROLLBACK.

Se você dominar tudo isso, você está tranquilamente no nível intermediário forte em SQL.

✦ **UNIQUE (Restrição de Unicidade)**

◆ **O que é**

A **constraint UNIQUE** é utilizada para garantir que **não existam valores duplicados em uma coluna ou conjunto de colunas** dentro de uma tabela.

Isso significa que dois registros **não podem ter o mesmo valor** naquele campo.

Essa restrição é muito usada para dados como:

- CPF
- Email
- Número de matrícula
- Código de produto

◆ **Exemplo simples**

```
CREATE TABLE funcionarios (  
  id_funcionario SERIAL PRIMARY KEY,  
  nome VARCHAR(100),  
  cpf VARCHAR(14) UNIQUE  
);
```

✦ Nesse caso:

- Cada funcionário pode ter **apenas um CPF único**.
- O banco não permitirá dois registros com o mesmo CPF.

◆ **UNIQUE em mais de uma coluna**

Também é possível garantir que **uma combinação de colunas não se repita**.

Exemplo em uma folha de pagamento:

```
ALTER TABLE folha_pagamento  
ADD CONSTRAINT pagamento_unico  
UNIQUE (id_funcionario, mes, ano);
```

✦ Isso garante que:

- Um funcionário **não possa ter dois pagamentos no mesmo mês e ano**.
-

🔴 RANK() – Função de Ranking

◆ O que é

A função RANK() é uma **função de janela (Window Function)** utilizada para **criar classificações (ranking)** dentro de uma consulta.

Ela permite ordenar registros e atribuir **uma posição numérica para cada linha**.

É muito usada em relatórios como:

- Ranking de salários
- Ranking de vendas
- Ranking de desempenho

◆ Exemplo de ranking salarial

```
SELECT  
RANK() OVER (ORDER BY salario DESC) AS ranking,  
nome,  
salario  
FROM funcionarios;
```

🔴 Explicação:

- ORDER BY salario DESC → ordena do maior salário para o menor
- RANK() → gera a posição no ranking

◆ Exemplo de resultado

ranking nome salario

1	Ana	7000
2	João	5500
3	Pedro	4200

◆ Ranking por departamento

Também é possível criar ranking **dentro de grupos**, como por departamento.

```
SELECT  
nome,  
id_departamento,  
salario,  
RANK() OVER (PARTITION BY id_departamento ORDER BY salario DESC) AS ranking  
FROM funcionarios;
```

✦ Aqui o ranking é reiniciado **dentro de cada departamento**.

TIPOS DE DADOS (DATA TYPES)

Os **tipos de dados** definem que tipo de informação cada coluna pode armazenar.

Escolher o tipo correto é muito importante para garantir **integridade dos dados e melhor desempenho**.

Tipos mais utilizados

Tipo	Descrição
INTEGER	números inteiros
SERIAL	inteiro auto incremento
VARCHAR(n)	texto com limite
TEXT	texto grande
NUMERIC	números decimais
BOOLEAN	verdadeiro ou falso
DATE	data
TIMESTAMP	data e hora

Exemplo

```
CREATE TABLE usuarios (  
  id SERIAL PRIMARY KEY,  
  nome VARCHAR(100),  
  email VARCHAR(200),  
  idade INTEGER,  
  ativo BOOLEAN,  
  data_criacao TIMESTAMP  
);
```

NULL

NULL representa **ausência de valor**.

⚠ NULL **não é zero**

⚠ NULL **não é vazio**

Exemplo

```
SELECT *  
FROM funcionarios  
WHERE comissao IS NULL;
```

Verificar valores que NÃO são NULL

```
SELECT *  
FROM funcionarios  
WHERE comissao IS NOT NULL;
```

BETWEEN

Utilizado para filtrar valores dentro de um intervalo.

Exemplo

```
SELECT *  
FROM funcionarios  
WHERE salario BETWEEN 2000 AND 5000;
```

Esse comando retorna salários **entre 2000 e 5000**.

IN

Permite verificar se um valor está dentro de uma lista.

Exemplo

```
SELECT *  
FROM clientes  
WHERE cidade IN ('São Paulo', 'Rio de Janeiro', 'Belo Horizonte');
```

Esse comando retorna clientes dessas cidades.

LIKE

Utilizado para **buscar padrões em texto**.

Caracteres especiais

Símbolo Significado

% qualquer sequência de caracteres

_ apenas um caractere

Exemplos

Buscar nomes que começam com "Jo":

```
SELECT *  
FROM clientes  
WHERE nome LIKE 'Jo%';
```

Buscar nomes que terminam com "son":

```
SELECT *  
FROM clientes  
WHERE nome LIKE '%son';
```

CONSTRAINTS (RESTRIÇÕES)

Constraints são regras aplicadas às colunas para garantir **integridade dos dados**.

NOT NULL

Impede valores nulos.

nome VARCHAR(100) NOT NULL

DEFAULT

Define valor padrão.

ativo BOOLEAN DEFAULT true

CHECK

Define regra de validação.

preco NUMERIC(10,2) CHECK (preco > 0)

Exemplo completo

```
CREATE TABLE produtos (  
  id SERIAL PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  preco NUMERIC(10,2) CHECK (preco > 0),  
  ativo BOOLEAN DEFAULT true  
);
```

INNER JOIN

Retorna apenas registros que possuem correspondência nas duas tabelas.

```
SELECT p.nome, c.nome  
FROM pedidos p
```

```
INNER JOIN clientes c
ON p.cliente_id = c.id;
```

LEFT JOIN

Retorna todos os registros da tabela da esquerda, mesmo que não exista correspondência na tabela da direita.

```
SELECT c.nome, p.id
FROM clientes c
LEFT JOIN pedidos p
ON c.id = p.cliente_id;
```

RIGHT JOIN

Retorna todos os registros da tabela da direita.

```
SELECT c.nome, p.id
FROM clientes c
RIGHT JOIN pedidos p
ON c.id = p.cliente_id;
```

DELETE vs TRUNCATE

DELETE

Remove registros **linha por linha**.

```
DELETE FROM funcionarios;
```

Pode usar WHERE:

```
DELETE FROM funcionarios
WHERE salario < 2000;
```

TRUNCATE

Remove **todos os registros da tabela rapidamente**.

```
TRUNCATE TABLE funcionarios;
```

⚠ Não pode usar WHERE.

CTE (COMMON TABLE EXPRESSION)

Uma **CTE** permite criar consultas temporárias dentro de uma query.

Utiliza a palavra **WITH**.

Exemplo

```
WITH media_salario AS (  
  SELECT AVG(salario) AS media  
  FROM funcionarios  
)
```

```
SELECT nome, salario  
FROM funcionarios, media_salario  
WHERE salario > media;
```

Esse comando lista funcionários **acima da média salarial**.

🚀 DENSE_RANK

Similar ao RANK, mas **não pula posições**.

Exemplo

```
SELECT  
nome,  
salario,  
DENSE_RANK() OVER (ORDER BY salario DESC) AS ranking  
FROM funcionarios;
```

🚀 EXPLAIN

O comando **EXPLAIN** mostra como o banco executa uma consulta.

Isso ajuda a entender **performance**.

Exemplo

```
EXPLAIN  
SELECT *  
FROM funcionarios  
WHERE nome = 'Carlos';
```

🚀 NORMALIZAÇÃO DE BANCO DE DADOS

A normalização organiza os dados para evitar **redundância e inconsistências**.

1ª Forma Normal (1FN)

Cada coluna deve ter **apenas um valor**.

✗ errado

telefone = "9999-1111 / 9999-2222"

✓ correto

Uma tabela separada de telefones.

2ª Forma Normal (2FN)

Todos os campos devem depender **da chave primária inteira**.

3ª Forma Normal (3FN)

Nenhum campo deve depender de outro campo que não seja a chave primária.